

JAVA COLLECTIONS FRAMEWORK

Complete Interview Revision Guide

Hierarchy • List / Set / Queue / Map • Internal Working
Iterators • Comparable vs Comparator • Fail-Fast vs Fail-Safe
Time Complexity • Interview Q&A • Spring Boot Relevance

. Table of Contents

- [01 Complete Collections Hierarchy Diagram](#)
- [02 List vs Set vs Queue vs Map — Core Comparison](#)
- [03 List Implementations: ArrayList vs LinkedList vs Vector](#)
- [04 Set Implementations: HashSet vs LinkedHashSet vs TreeSet](#)
- [05 Map Implementations: HashMap vs LinkedHashMap vs TreeMap vs Hashtable vs ConcurrentHashMap](#)
- [06 Internal Working: HashMap Buckets, Collisions, hashCode\(\) & equals\(\)](#)
- [07 Iterator vs ListIterator vs Enumeration](#)
- [08 Comparable vs Comparator](#)
- [09 Fail-Fast vs Fail-Safe Iterators](#)
- [10 Time Complexity Master Tables](#)
- [11 Frequently Asked Interview Questions](#)
- [12 Spring Boot Interview Relevance](#)
- [13 Memory Tricks & One-Liners](#)
- [14 Note on 'Previously Discussed Questions'](#)

1. Complete Collections Hierarchy Diagram

The Java Collections Framework splits into two root interfaces: **Collection** (List, Set, Queue) and **Map** (which is NOT a child of Collection). Below is the full hierarchy tree used most often in interviews.

```
Iterable (interface)
  |-- Collection (interface)
    |-- List (interface) – ordered, duplicates allowed
      |-- ArrayList
      |-- LinkedList (also implements Deque)
      |-- Vector (legacy, synchronized)
      |   |-- Stack (legacy, extends Vector)
      |-- CopyOnWriteArrayList (concurrent)
    |-- Set (interface) – no duplicates
      |-- HashSet – unordered, backed by HashMap
      |   |-- LinkedHashSet – insertion order
      |-- SortedSet (interface)
        |-- NavigableSet (interface)
          |-- TreeSet – sorted (Red-Black Tree)
    |-- Queue (interface) – FIFO processing
      |-- PriorityQueue – heap ordered
      |-- LinkedList (dual role: List + Deque)
      |-- Deque (interface) – double-ended queue
        |-- ArrayDeque – resizable array, no nulls
        |-- LinkedBlockingDeque (concurrent)

Map (interface) – separate root, key-value pairs, NOT a Collection
  |-- HashMap – unordered, allows one null key
  |   |-- LinkedHashMap – insertion/access order
  |-- Hashtable (legacy, synchronized, no nulls)
  |   |-- Properties (legacy, String key/value)
  |-- SortedMap (interface)
    |-- NavigableMap (interface)
      |-- TreeMap – sorted (Red-Black Tree)
  |-- ConcurrentMap (interface)
    |-- ConcurrentHashMap – thread-safe, segment/bin locking
```

Interview one-liner: "Collection is for single elements (List/Set/Queue), Map is for key-value pairs. Map does NOT extend Collection — that's a classic trick question."

2. List vs Set vs Queue vs Map — Core Comparison

Aspect	List	Set	Queue	Map
Duplicates	Allowed	Not allowed	Allowed (impl. dependent)	Duplicate keys not allowed; values can repeat
Order	Insertion order maintained, index-based	No guaranteed order (except Linked/Tree variants)	FIFO / priority based on impl.	Depends on impl. (Hash=none, Linked=insertion, Tree=sorted)
Null elements	Multiple nulls allowed (ArrayList)	One null allowed (HashSet)	ArrayDeque: no null; PriorityQueue: no null	HashMap: 1 null key, many null values
Access	By index — get(i)	No index access, only iteration	peek/poll from head	By key — get(key)
Key implementations	ArrayList, LinkedList, Vector	HashSet, LinkedHashSet, TreeSet	PriorityQueue, ArrayDeque, LinkedList	HashMap, LinkedHashMap, TreeMap, Hashtable, ConcurrentHashMap
Typical use case	Ordered collection, frequent reads by index	Uniqueness enforcement, membership tests	Task scheduling, BFS, producer-consumer	Fast key-based lookup, caching, indexing data
Extends	Collection	Collection	Collection	Standalone root interface

Interview one-liner: "List = ordered + duplicates, Set = unique, Queue = processing order (usually FIFO), Map = key-value pairs."

3. List Implementations: ArrayList vs LinkedList vs Vector

Feature	ArrayList	LinkedList	Vector
Underlying structure	Dynamic resizable array	Doubly linked list (nodes with prev/next)	Dynamic resizable array
Access time — get(i)	O(1) — direct index access	O(n) — traverse from head/tail	O(1) — direct index access
Insert/Delete at end	O(1) amortized	O(1)	O(1) amortized
Insert/Delete at middle/start	O(n) — shifts elements	O(1) once position found (O(n) to find it)	O(n) — shifts elements
Memory overhead	Low — only data + unused capacity	High — each node stores data + 2 pointers	Low — similar to ArrayList
Thread safety	Not synchronized	Not synchronized	Synchronized (every method locked)
Growth strategy	Grows by 50% of current size	No resizing needed (node-based)	Grows by 100% (doubles) by default
Implements	List, RandomAccess	List, Deque, Queue	List, RandomAccess
Advantages	Fast random access; cache-friendly (contiguous memory); low memory per element	Fast insert/delete at ends; efficient as Queue/Deque/Stack; no resizing cost	Thread-safe out of the box; legacy API compatibility
Disadvantages	Slow insert/delete in middle; resizing costs occasionally	Slow random access; higher memory per node; poor cache locality	Synchronization overhead makes it slower than ArrayList; considered legacy
Real-time usage	Storing/display lists, search results, DTO lists in REST APIs	Implementing LRU cache, undo/redo history, task queues (as Deque)	Rarely used today; seen in legacy codebases; replaced by Collections.synchronizedList() or CopyOnWriteArrayList
When to pick	Read-heavy, index-based access	Write-heavy at ends, frequent add/remove, queue/stack behavior	Only for legacy compatibility — otherwise avoid

Interview one-liner: "ArrayList = fast read, slow middle-insert. LinkedList = fast insert/delete at ends, slow read. Vector = ArrayList's synchronized, legacy cousin — nobody uses it in new code."

4. Set Implementations: HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Underlying structure	HashMap internally (element = key, dummy value)	LinkedHashMap internally (hash table + doubly linked list)	TreeMap internally (Red-Black self-balancing tree)
Ordering	No order guaranteed	Insertion order preserved	Sorted order (natural or Comparator)
add/remove/contains	O(1) average	O(1) average	O(log n)
Null elements	One null allowed	One null allowed	No null (throws NPE if natural ordering used)
Extra memory	Lowest	Slightly higher (linked list pointers)	Higher (tree node pointers: left, right, parent)
Interfaces implemented	Set, Collection	Set, Collection (extends HashSet)	NavigableSet, SortedSet
Special methods	—	—	first(), last(), ceiling(), floor(), headSet(), tailSet()
Advantages	Fastest lookup/insert for uniqueness checks	Predictable iteration order + fast lookup	Always sorted; range queries; navigable methods
Disadvantages	Iteration order unpredictable/unstable across runs	Slightly slower than HashSet due to linked list upkeep	Slowest of the three (O(log n)); requires Comparable/Comparator
Real-time usage	De-duplicating IDs, membership checks (e.g. blacklist lookup)	Maintaining recently-used unique items, LRU-style sets, caching layers where order matters	Leaderboards, range-based queries (e.g. price between X-Y), sorted unique tags
When to pick	You only care about uniqueness, not order	You need uniqueness + insertion order	You need uniqueness + sorted order or range operations

Interview one-liner: "HashSet = fast, unordered. LinkedHashSet = fast + remembers insertion order. TreeSet = sorted, but O(log n) — pick it only when you need order."

5. Map Implementations: HashMap vs LinkedHashMap vs TreeMap vs Hashtable vs ConcurrentHashMap

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable	ConcurrentHashMap
Ordering	No order	Insertion order (or access order if configured)	Sorted by key	No order	No order
Thread safety	Not synchronized	Not synchronized	Not synchronized	Synchronized (whole map locked)	Thread-safe (bucket/segment level locking, CAS)
Null keys/values	1 null key, multiple null values	1 null key, multiple null values	No null key (NPE); null values ok	No null key or value	No null key or value
get/put complexity	O(1) average, O(log n) worst-case (Java 8+ treeified buckets)	O(1) average	O(log n) always	O(1) average	O(1) average
Internal structure	Array of buckets + LinkedList/Red-Black Tree (JDK 8+)	HashMap + doubly linked list across entries	Red-Black self-balancing tree	Array of buckets + LinkedList (legacy style)	Array of nodes + CAS + synchronized blocks per bin (JDK 8+)
Iteration performance	Fast	Fast, predictable order	Slower (tree traversal) but sorted	Slow (legacy overhead)	Fast, weakly consistent iterator
Advantages	Best general-purpose performance for lookups	Predictable order + HashMap speed; great for LRU cache (via removeEldestEntry)	Sorted keys; range queries; navigable methods	Simple thread safety (legacy)	High-performance thread safety without locking the whole map
Disadvantages	Not thread-safe; unordered	Slightly more memory/overhead than HashMap	Slower writes/reads O(log n); needs Comparable/Comparator	Poor performance under concurrency (single global lock); legacy	Slightly more complex; weakly consistent iteration (may not reflect very latest updates)
Real-time usage	Caching, counting frequencies, general key-value storage	LRU cache implementation, maintaining config/property order	Sorted dictionaries, range-based reports, time-series keyed by date	Rare — only legacy systems	Shared caches in multi-threaded services, in-memory counters in Spring Boot apps, session stores
When to pick	Default choice for single-threaded key-value storage	Need order + map speed	Need sorted keys or range operations	Avoid in new code	Multi-threaded environments needing a map

Interview one-liner: "HashMap = fast & unsafe. LinkedHashMap = fast + ordered. TreeMap = sorted but slower. Hashtable = legacy synchronized (avoid). ConcurrentHashMap = modern thread-safe choice."

6. Internal Working: HashMap Buckets, Collisions, hashCode() & equals()

How HashMap.put(key, value) works internally:

- Java computes `key.hashCode()`, then applies a supplemental hash spreading function: `hash ^ (hash >>> 16)` — this reduces collisions from poor `hashCode()` implementations.
- The final hash is mapped to a bucket index using `(n - 1) & hash`, where `n` = table capacity (always a power of 2).
- If the bucket is empty, the new node is placed directly.
- If the bucket already has entries (a **collision**), Java checks each existing key using `equals()`. If a match is found, the value is updated; otherwise the new entry is appended to that bucket's linked list.
- **Since Java 8:** if a single bucket's linked list grows beyond 8 nodes AND table capacity ≥ 64 , that bucket is converted ("treeified") into a Red-Black Tree, changing worst-case lookup from $O(n)$ to $O(\log n)$.
- When the map's size exceeds `capacity × loadFactor` (default load factor = 0.75), the table resizes (doubles) and all entries are rehashed into new bucket positions.

Why both hashCode() and equals() must be overridden together:

- `hashCode()` decides *which bucket* an object goes into; `equals()` decides *which entry within that bucket* is a logical match.
- **Contract:** if two objects are equal via `equals()`, they **MUST** return the same `hashCode()`. The reverse is not required (different equal-check but same `hashCode` is allowed — that's a collision, not a bug).
- If you override `equals()` but not `hashCode()`, two 'equal' objects can land in different buckets — duplicate entries get inserted into a `HashSet/HashMap` even though `equals()` says they're the same. This is the #1 interview trap.
- Immutable fields should be used for `hashCode()/equals()` (e.g. don't build a hash from a mutable list field) — otherwise an object can 'disappear' from a `HashMap` after its hash-relevant field changes post-insertion.

Collision handling summary:

Scenario	Behavior
Different hashCode, different bucket	No collision — $O(1)$ direct access
Same hashCode, different bucket range (before mask)	Spreading function reduces this; still possible with mask collision
Same bucket index (true collision)	Nodes chained via linked list (or tree if > 8 nodes & capacity ≥ 64)
Same hashCode, <code>equals()</code> returns false	Both entries stored in same bucket as separate keys
Same hashCode, <code>equals()</code> returns true	Treated as same key — value is overwritten

Interview one-liner: "*hashCode() picks the bucket, equals() picks the entry inside the bucket. Break the contract and your HashSet silently stores duplicates.*"

7. Iterator vs ListIterator vs Enumeration

Feature	Iterator	ListIterator	Enumeration
Applicable to	All Collection types (List, Set, Queue, Map via keySet/values)	Only List implementations	Legacy classes: Vector, Hashtable
Traversal direction	Forward only	Both forward and backward	Forward only
Modification support	remove() only	add(), set(), remove() — full modification during iteration	No modification allowed
Index access	Not available	nextIndex(), previousIndex() available	Not available
Fail-fast?	Yes (in most collections)	Yes	No — it's fail-safe (legacy, doesn't check for concurrent modification)
Introduced in	Java 1.2	Java 1.2	Java 1.0 (oldest, legacy)
Real-time usage	General-purpose safe traversal + removal while iterating	Editing a list in place while walking it (e.g. replacing values), reverse traversal	Rare — only when working with legacy Vector/Hashtable APIs

Interview one-liner: "Iterator walks forward and can only remove. ListIterator walks both ways and can add/set/remove. Enumeration is the fossil from Java 1.0 you'll only see in legacy code."

8. Comparable vs Comparator

Aspect	Comparable	Comparator
Package	java.lang	java.util
Method to implement	compareTo(Object o)	compare(Object o1, Object o2)
Where defined	Inside the class whose objects are compared (intrusive)	Separate class or lambda, outside the object's own class (non-intrusive)
Number of sorting sequences	Only one natural ordering per class	Multiple — write as many Comparators as needed
Effect on original class	Requires modifying the class itself	No modification needed — plug in externally
Usage call	Collections.sort(list)	Collections.sort(list, comparator)
Java 8 style	obj implements Comparable<T>	Comparator.comparing(Class::getField).thenComparing(...)
Advantages	Simple, built-in default sort behavior for a type	Flexible — sort same objects by different fields/orders without touching the class
Disadvantages	Locked to one ordering; changing it affects every use of that class	Slightly more boilerplate; must remember to pass comparator everywhere
Real-time usage	Sorting Employee objects by employeeid as the 'default' natural order	Sorting the same Employee list by name in one screen and by salary in another, e.g. in REST API sort params

Interview one-liner: "Comparable = 'I know how to compare myself' (one way, inside the class). Comparator = 'here's an outside rule for comparing you' (many ways, outside the class)."

9. Fail-Fast vs Fail-Safe Iterators

Aspect	Fail-Fast	Fail-Safe
Behavior on modification	Throws <code>ConcurrentModificationException</code> immediately if the collection is structurally modified during iteration	Iterates over a clone/snapshot, so it does NOT throw an exception on concurrent modification
Mechanism	Uses an internal <code>modCount</code> counter; iterator checks it on every <code>next()</code>	Iterates a separate copy of the underlying data (or uses copy-on-write semantics)
Memory usage	Low — no extra copy	Higher — a clone/snapshot is created
Data freshness	Always reflects the live collection	May not reflect the most recent updates (works on a snapshot)
Example classes	<code>ArrayList</code> , <code>HashMap</code> , <code>HashSet</code> , <code>Vector</code> (with plain <code>Iterator</code>)	<code>CopyOnWriteArrayList</code> , <code>ConcurrentHashMap</code> , <code>CopyOnWriteArraySet</code>
Advantages	Immediately surfaces bugs from unsafe concurrent access; predictable, live data	Safe for concurrent read/write without exceptions; good for read-heavy multi-threaded access
Disadvantages	Not safe for multi-threaded modification during iteration; can crash the app if not careful	Extra memory/CPU cost of copying; iterator may show stale data
Real-time usage	Typical single-threaded application logic — the exception protects you from silent bugs	Multi-threaded caches/event listener lists where reads vastly outnumber writes (e.g. <code>CopyOnWriteArrayList</code> for observer lists)

Interview one-liner: "Fail-fast throws `ConcurrentModificationException` on a live list (`ArrayList`, `HashMap`). Fail-safe iterates a snapshot and never throws (`CopyOnWriteArrayList`, `ConcurrentHashMap`)."

10. Time Complexity Master Tables

List implementations:

Operation	ArrayList	LinkedList	Vector
get(index)	O(1)	O(n)	O(1)
add(element) — at end	O(1) amortized	O(1)	O(1) amortized
add(index, element)	O(n)	O(n) to find + O(1) to insert	O(n)
remove(index)	O(n)	O(n) to find + O(1) to remove	O(n)
contains(element)	O(n)	O(n)	O(n)
Iteration	O(n)	O(n)	O(n)

Set implementations:

Operation	HashSet	LinkedHashSet	TreeSet
add	O(1) avg	O(1) avg	O(log n)
remove	O(1) avg	O(1) avg	O(log n)
contains	O(1) avg	O(1) avg	O(log n)
Iteration	O(n)	O(n)	O(n)

Map implementations:

Operation	HashMap	LinkedHashMap	TreeMap	Hashtable	ConcurrentHashMap
get(key)	O(1) avg, O(log n) worst	O(1) avg	O(log n)	O(1) avg	O(1) avg
put(key, val)	O(1) avg, O(log n) worst	O(1) avg	O(log n)	O(1) avg	O(1) avg
remove(key)	O(1) avg	O(1) avg	O(log n)	O(1) avg	O(1) avg
containsKey	O(1) avg	O(1) avg	O(log n)	O(1) avg	O(1) avg

Queue / Deque implementations:

Operation	ArrayDeque	LinkedList	PriorityQueue
offer/add	O(1) amortized	O(1)	O(log n)
poll/remove (head)	O(1)	O(1)	O(log n)
peek	O(1)	O(1)	O(1)
contains	O(n)	O(n)	O(n)

Interview one-liner: "Array-backed = O(1) read, O(n) middle write. Linked-backed = O(1) end write, O(n) read. Tree-backed = O(log n) everything, but sorted."

11. Frequently Asked Interview Questions

Q1. Why is Map not a part of the Collection interface?

Because Map stores key-value pairs, not single elements. Its contract (get by key, no duplicate keys) doesn't fit Collection's single-element contract (add, iterate, contains-element). Map is deliberately a separate root interface.

Q2. How does HashMap handle collisions?

By chaining colliding entries in a linked list inside the same bucket. Since Java 8, if a bucket's chain exceeds 8 nodes (and table capacity ≥ 64), it's converted into a Red-Black Tree for $O(\log n)$ worst case instead of $O(n)$.

Q3. What is the default capacity and load factor of a HashMap?

Default initial capacity is 16, default load factor is 0.75. The map resizes (doubles capacity) once size exceeds $\text{capacity} \times \text{loadFactor}$ (12 entries for the default 16-capacity map).

Q4. Why should hashCode() and equals() always be overridden together?

Because HashMap/HashSet use hashCode() to locate the bucket and equals() to confirm identity within it. Breaking the contract causes duplicate 'equal' objects to be stored separately, or valid lookups to fail.

Q5. Difference between HashMap and ConcurrentHashMap?

HashMap is not thread-safe and can even loop infinitely / corrupt data under concurrent modification. ConcurrentHashMap achieves thread safety via bucket-level locking + CAS operations (Java 8+), allowing high-concurrency reads/writes without locking the entire map.

Q6. Why is String often used as a HashMap key?

Strings are immutable, so their hashCode() is stable and can be cached (computed once). This guarantees the key's bucket location never changes after insertion — critical for correctness.

Q7. What happens if you use a mutable object as a HashMap key and later modify it?

The object's hashCode() may change, so it now maps to a different bucket than the one it was originally stored in. The entry effectively becomes unreachable via get() even though it's still physically in the map — a memory leak / silent bug.

Q8. ArrayList vs LinkedList — which would you use for a stack-like structure?

Neither, ideally — use ArrayDeque, which is faster than both Stack (legacy, synchronized) and LinkedList for stack/queue operations due to better cache locality and no synchronization overhead.

Q9. How does TreeMap/TreeSet maintain sorted order internally?

Both are backed by a Red-Black Tree (a self-balancing binary search tree), guaranteeing $O(\log n)$ for insert, delete, and search while keeping keys always sorted (natural order or supplied Comparator).

Q10. Difference between Comparable and Comparator — when would you use each?

Comparable defines a class's single 'natural' ordering (compareTo, inside the class). Comparator defines external, pluggable orderings (compare, outside the class) — use it when you need multiple sort criteria or can't modify the original class.

Q11. What is the difference between fail-fast and fail-safe iterators?

Fail-fast iterators (`ArrayList`, `HashMap`) throw `ConcurrentModificationException` if the collection is structurally modified during iteration by another thread/thread-unsafe code. Fail-safe iterators (`CopyOnWriteArrayList`, `ConcurrentHashMap`) iterate over a snapshot/clone and never throw, but may show stale data.

Q12. Why is Vector considered legacy? What replaced it?

`Vector` synchronizes every single method, causing performance overhead even in single-threaded use. It's been effectively replaced by `ArrayList` (single-threaded) or `Collections.synchronizedList()` / `CopyOnWriteArrayList` (multi-threaded).

Q13. How does LinkedHashMap implement LRU Cache?

By constructing it with `accessOrder=true`, entries are reordered to the end of the internal linked list on every access (not just insertion). Overriding `removeEldestEntry()` to return true once a size threshold is exceeded evicts the least-recently-used entry automatically.

Q14. What's the initial capacity/growth rule difference between ArrayList and Vector?

`ArrayList` grows by roughly 50% of current capacity when full; `Vector` doubles (100% growth) by default. This makes `ArrayList` slightly more memory-efficient for gradually growing collections.

Q15. Can you have a HashMap with a null key? What about Hashtable?

`HashMap` allows exactly one null key and multiple null values. `Hashtable` and `ConcurrentHashMap` throw `NullPointerException` for null keys or values — a common trick question.

12. Spring Boot Interview Relevance

Collections show up constantly in Spring Boot interviews — not as trivia, but as the backbone of how the framework and typical applications are built. Key connections to know:

Spring Boot Context	Collection Used	Why It Matters
Bean definitions registry (BeanFactory)	ConcurrentHashMap	Spring's internal singleton bean cache is a ConcurrentHashMap for thread-safe bean lookup during concurrent request handling.
@Autowired List<T> / Map<String,T> injection	List, Map	Spring auto-collects all beans of a type into a List, or by bean name into a Map<String,T> — used heavily for strategy pattern implementations (e.g. multiple PaymentProcessor beans).
Request-scoped caching (@Cacheable backing store)	ConcurrentHashMap / LinkedHashMap (via Caffeine/Guava)	In-memory cache implementations commonly use ConcurrentHashMap for thread safety or LinkedHashMap for LRU eviction policies.
JPA/Hibernate entity collections (@OneToMany)	List, Set	Choosing List vs Set for @OneToMany affects duplicate handling and query performance (Set avoids duplicate rows from JOIN fetches).
REST controller responses (JSON arrays/objects)	List, Map	Jackson serializes List → JSON array, Map → JSON object; ordering matters when using LinkedHashMap for predictable API responses.
Configuration properties binding (@ConfigurationProperties)	List, Map, Set	Spring Boot binds YAML/properties list/map syntax directly into typed List/Map fields.
Concurrent request counters / rate limiting	ConcurrentHashMap, AtomicInteger values	Thread-safe counters per client/IP in filters/interceptors rely on ConcurrentHashMap to avoid race conditions.
Sorting API results (e.g. Pageable, Sort)	Comparator	Spring Data's Sort/Pageable APIs are effectively built on Comparator-style ordering logic passed to repository queries.
Event listener registries	CopyOnWriteArrayList	Application event multicasters often use CopyOnWriteArrayList for thread-safe listener iteration during publish.

Interview one-liner: "Spring itself is a huge consumer of the Collections Framework — bean registries use ConcurrentHashMap, multi-bean injection uses List/Map, and caching layers lean on LinkedHashMap or ConcurrentHashMap."

13. Memory Tricks & Interview One-Liners

Quick mnemonic — 'A-L-V' for Lists:

- **ArrayList** → **A**rray → fast Access by index
- **LinkedList** → **L**inks → fast at edges (head/tail)
- **Vector** → **V**intage/synchronized → avoid in new code

Quick mnemonic — Sets remember their name:

- **HashSet** = 'Hashed, unordered' — think random bucket order
- **LinkedHashSet** = 'Hash speed + Linked memory' — insertion order preserved
- **TreeSet** = 'Tree = sorted branches' — always sorted, $O(\log n)$

Quick mnemonic — Maps mirror the Sets:

- **HashMap** ↔ **HashSet** (HashSet is literally backed by a HashMap internally)
- **LinkedHashMap** ↔ **LinkedHashSet** — same insertion-order trick
- **TreeMap** ↔ **TreeSet** — same Red-Black Tree under the hood
- **Hashtable** = 'the ancestor no one invites to the party' — legacy, fully synchronized
- **ConcurrentHashMap** = 'the modern, polite version of Hashtable' — thread-safe without locking everything

One-liners to say out loud in interviews:

"List = ordered + duplicates. Set = unique. Queue = processing order. Map = key-value."

"ArrayList reads fast, LinkedList inserts/removes fast at the ends."

"hashCode() finds the bucket, equals() finds the exact entry."

"Fail-fast throws on a live list; fail-safe quietly works on a snapshot."

"Comparable is built-in and singular; Comparator is external and plural."

"HashMap is fast but unsafe; ConcurrentHashMap is fast AND safe."

"TreeSet/TreeMap trade $O(1)$ speed for guaranteed sorted order — $O(\log n)$ is the price of sorting."

"Map is NOT a Collection — it's a parallel root interface for key-value pairs."

14. Note on 'Previously Discussed Questions'

You asked me to include the questions we'd already discussed together, along with your mistakes and corrections. I don't actually have access to earlier conversations — each chat starts fresh unless memory is turned on in your settings, and this conversation doesn't show any prior Collections Q&A between us.

If you'd like that material included, you have two options:

- **Paste in the questions and your answers/mistakes** here in the chat, and I'll add a dedicated 'Your Practice Log' section to this PDF with corrections explained alongside each one.
- **Turn on Claude's memory** (Settings → 'Search and reference past chats' / 'Generate memory from chat history'), so future sessions like this one can automatically pull in what we've covered before.

Everything else you asked for — the hierarchy diagram, all five comparison tables, internal HashMap mechanics, iterators, Comparable vs Comparator, fail-fast vs fail-safe, time complexity, interview Q&A, Spring Boot relevance, and memory tricks — is covered in the sections above.